

대본

안녕하세요 이번에 3단계 결과 발표를 맡게 된 22학번 이강석, 24학번 권기영입니다.

(넘기며)

이러한 순서로 발표를 진행하겠습니다.

1. 프로젝트 개요

먼저 프로젝트 개요입니다.

본 프로젝트의 목표는 ucode를 어셈블하고 실행할 수 있는 GUI와 CLI 인터프리터를 개발하는 것입니다. 프로젝트를 추진한 이유는, ucode가 스택 기반 중간 언어라 이해하기 어렵고, 기존 CLI 환경에서는 실행 흐름을 직관적으로 확인하기 힘들기 때문입니다. 그래서 저희는 실행 흐름을 한눈에 확인할 수 있는 프로그램을 제공하고자 하였습니다. 또한 Applmage 형태로 배포하여, 별도의 설정 없이 다양한 리눅스 환경에서 바로 실행할 수 있는 편의성도 확보하였습니다.

2. 프로그램 구조 및 기능

다음으로 프로그램의 구조와 기능입니다.

본 프로그램은 CLI와 GUI 두 가지 인터페이스를 제공합니다. 두 인터페이스 모두 내부적으로 Assembler, Runner, IO 유틸 모듈을 호출해 동작합니다. 또한 실행 상태 관리는 Record 데이터 모델을 사용하는데 상태를 기록하는 건 Runner가 담당하며, 그 저장된 상태를 이용해 현재 실행 상태를 복원하는 건 GUI가 담당합니다.

3. 변경점

다음으로 설계와 달라진 점을 설명하겠습니다.

먼저 실행 엔진을 구현하면서 duplicate 명령어가 없고, 전역 프레임의 end 명령어가 어셈블러에서 무시되는 문제가 있었습니다. 특히 함수 호출에서는 함수 프레임 기초가 먼저 만들어지고 그 위에 인자가 쌓이는 구조였기 때문에, call 명령어 시점 기준으로 프레임 기초의 위치를 판단하기 어려운 구조적 문제가 있었습니다.

이를 해결하기 위해 duplicate 명령어를 추가하고 end를 return으로 매핑했으며, 프레임 위치를 명확히 식별되도록 인자를 먼저 쌓는 방식으로 변경했습니다. 그러나 이 방식에

서도 return 시 인자 개수를 알기 어려워 회수가 불안정했기 때문에, 어셈블러가 인자 개수를 자동으로 return의 피연산자로 부여하고 사용자를 위해 offset도 재계산하도록 개선하여 구조를 안정화했습니다.

다음으로 배포입니다. 리눅스 환경에서 배포하는 방식은 이렇게 있는데요. 저희는 다양한 리눅스 배포판 환경에서 동일하게 실행될 수 있도록 단일 파일 기반 AppImage 포맷을 채택해, 사용자가 별도 설치 없이 바로 실행 가능한 형태로 배포했습니다.

4. 데모

(pdf 예제 cli, 버블 정렬 gui)

데모에서는 먼저 CLI 환경으로 pdf 예제를 실행해보겠습니다.

(실행하고 입력하기)

보시는 것과 같이 기대한 결과가 출력되고 LST 파일 생성 메시지가 출력됩니다.

LST 파일을 보시면 원본 코드, 어셈블 결과 통계까지 있는 것을 확인할 수 있습니다.

다음은 GUI 환경으로 버블 정렬을 실행 해 보겠습니다.

(버블 정렬 하기)

슬라이드와 버튼으로 각 단계별 상황을 볼 수도 있고, 플레이 버튼으로 자동으로 진행되도록도 가능합니다. 또 통계, lst 도움말 등을 확인 할 수 있습니다.

5. 결과 분석

다음은 결과 분석입니다.

테스트를 위해 PDF 예제 코드외에도 버블 정렬, 이진 트리 DFS, 피보나치, 그리고 최대 공배수(GCD) 프로그램을 작성해 테스트 해보았습니다. 각 프로그램은 명령어나 안정성을 검사해볼 수 있도록 결과는, 이 프로그램들을 가지고 여러 경계값들로 테스트 해 보았을 때 기대한대로 실행되는 것을 확인할 수 있었습니다.

또한 이 중에서 버블정렬, DFS, 피보나치는 C, Python, Java로도 작성하여 실행 속도 비교해 보았습니다. 이때 공정한 비교를 위해서 인터프리터의 snapshot 및 change 기능 등 실행에 필수적이지 않은 기능은 모두 비활성화한 상태에서 테스트했습니다

실험 결과, 각각 2번째 3번째 4번째로 실행 속도가 빨랐고, 마지막 피보나치에서는 다른 언어들과 차이가 많이 나는 모습을 보여줬습니다.

이러한 결과가 나온 이유를 분석해보았을 때, 저희 인터프리터는 비교적 간단한 가상 머신 구조를 사용하고 있고 단순 연산 추가적인 처리들이 없어, 단순 연산이 빠르기 때문에 이것이 단순 연산이 많은 버블 정렬과 비교적 얇은 재귀 호출이 일어나는 DFS에서 좋은 영향을 주는 것으로 보입니다. 반대로 저희 인터프리터에서 함수 호출은 여러 처리를 거쳐 비용이 크기 때문에 재귀 호출이 많이 일어나는 피보나치에서는 실행 시간에 큰 영향을 주고, 피보나치 만큼 함수 호출이 많지는 않지만 DFS에서도 함수 호출이 많이 일어나기 때문에 여기에도 영향을 주어 그래프와 같은 결과가 나온 것 같습니다.

6. 향후 개선 방안

다음은 향후 개선 방안입니다.

현재 저희 인터프리터에는 Snapshot과 Change로 인한 메모리 과다 사용과 성능 저하가 있습니다. 그래서 주기적 파일저장, 스레드 분리 등을 통해서 이를 최적화 하는 방안을 생각해 보았습니다. 또 사용성 측면에서도 현재 간단한 디버그 메시지만 제공하기 때문에 오타 수정 제안, 중단점 등의 다양한 기능을 추가하는 방안도 생각해 보았습니다.

이상으로 발표를 마치겠습니다.